

Closure in JavaScript



Name: Matthew Kamal Saad

Submitted to: Eng. Ahmed Saber

Date: 23/1/2026

Introduction to JavaScript Closures

Closures are one of the most powerful and important concepts in JavaScript. They allow functions to access variables from an outer function even after the outer function has finished executing.

What is a Closure?

A closure is created when a function is defined inside another function, and the inner function retains access to the outer function's variables.

Why Closures are Important

Closures help in data encapsulation, maintaining state, and creating private variables. They are widely used in modern JavaScript frameworks and libraries.

How Closures Work

When a function is created, JavaScript stores the function together with its lexical environment. This environment consists of any local variables that were in scope at the time the closure was created.

Practical Examples of Closures Using JavaScript

Example 1: A function that remembers a variable value even after execution.

Example 2: Using closures for counters.

Example 3: Data hiding using closures.

Common Use Cases of Closures

- Event handlers
- Callbacks and asynchronous programming
- Private variables
- Function factories

Advantages and Disadvantages

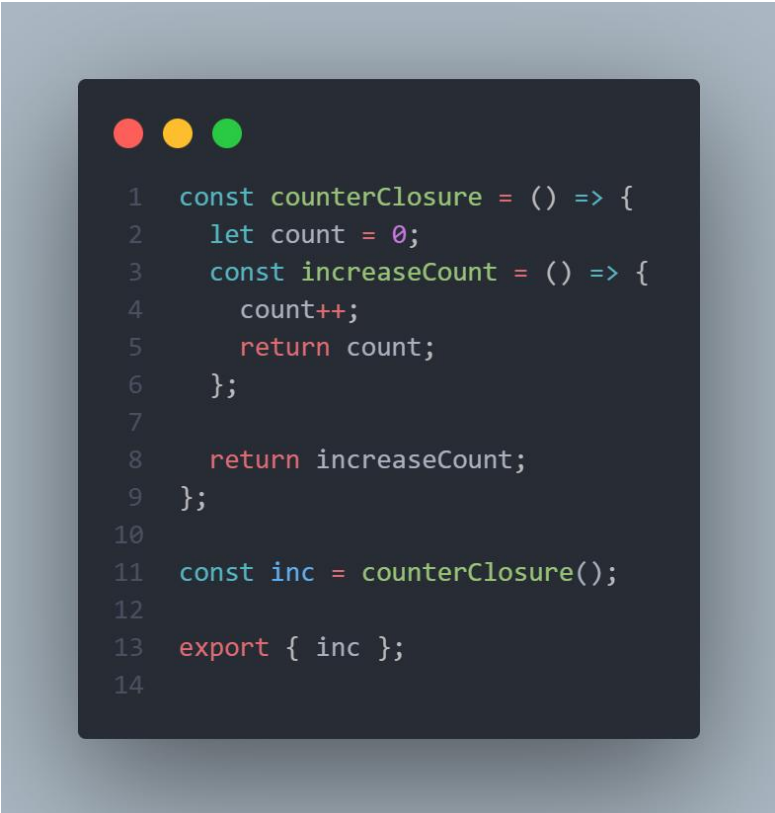
Advantages:

- Data encapsulation
- Better control over variables

Disadvantages:

- Increased memory usage
- Can make code harder to understand if misused

Practical Examples of Closures Using JavaScript in Visual Studio



```
1  const counterClosure = () => {  
2    let count = 0;  
3    const increaseCount = () => {  
4      count++;  
5      return count;  
6    };  
7  
8    return increaseCount;  
9  };  
10  
11  const inc = counterClosure();  
12  
13  export { inc };  
14
```

References

https://www.w3schools.com/js/js_closures.asp

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Closures>